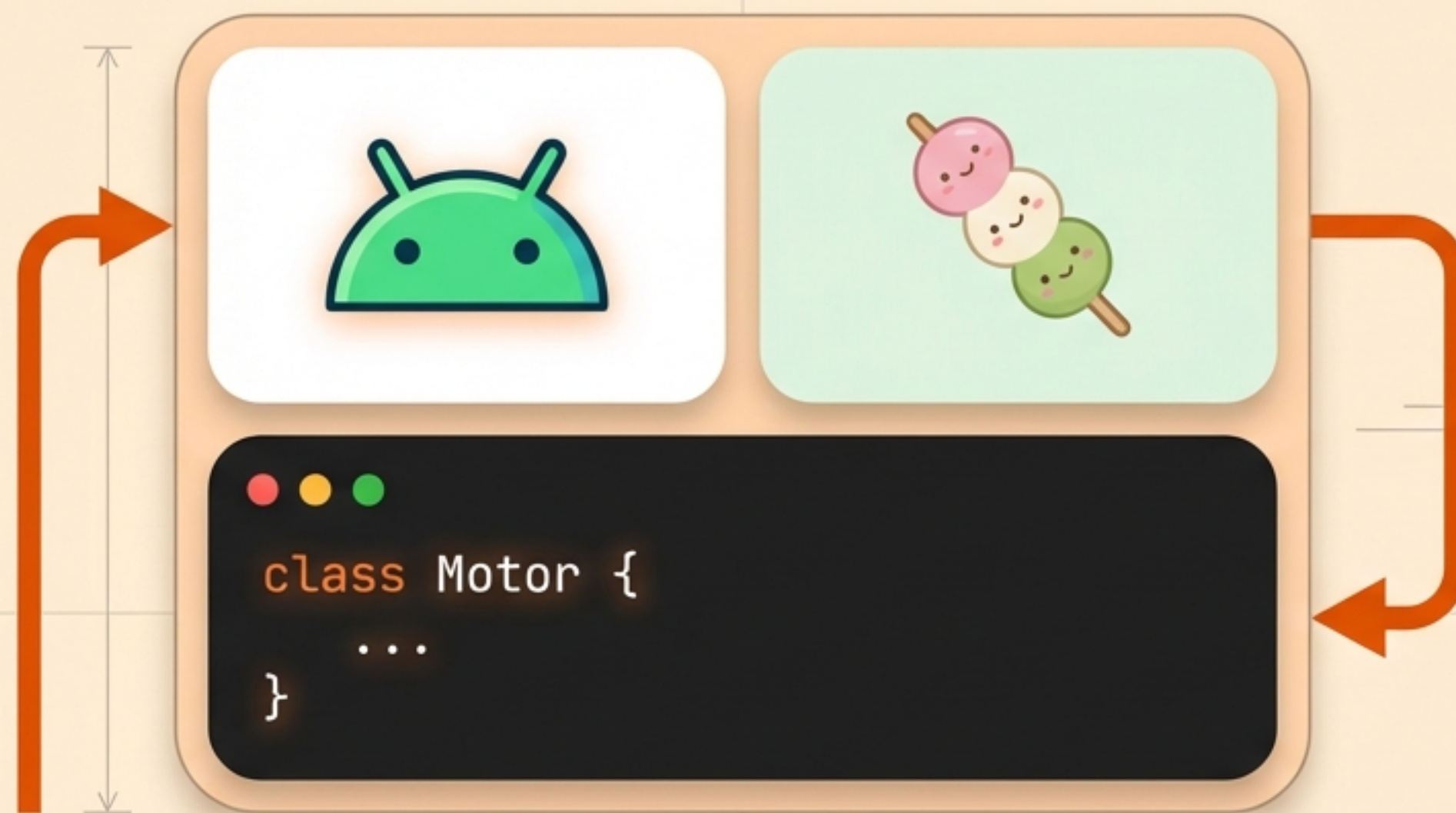




# Construyendo un Universo Mochi con Jetpack Compose

De un lienzo en blanco a un motor de físicas en 3 niveles.

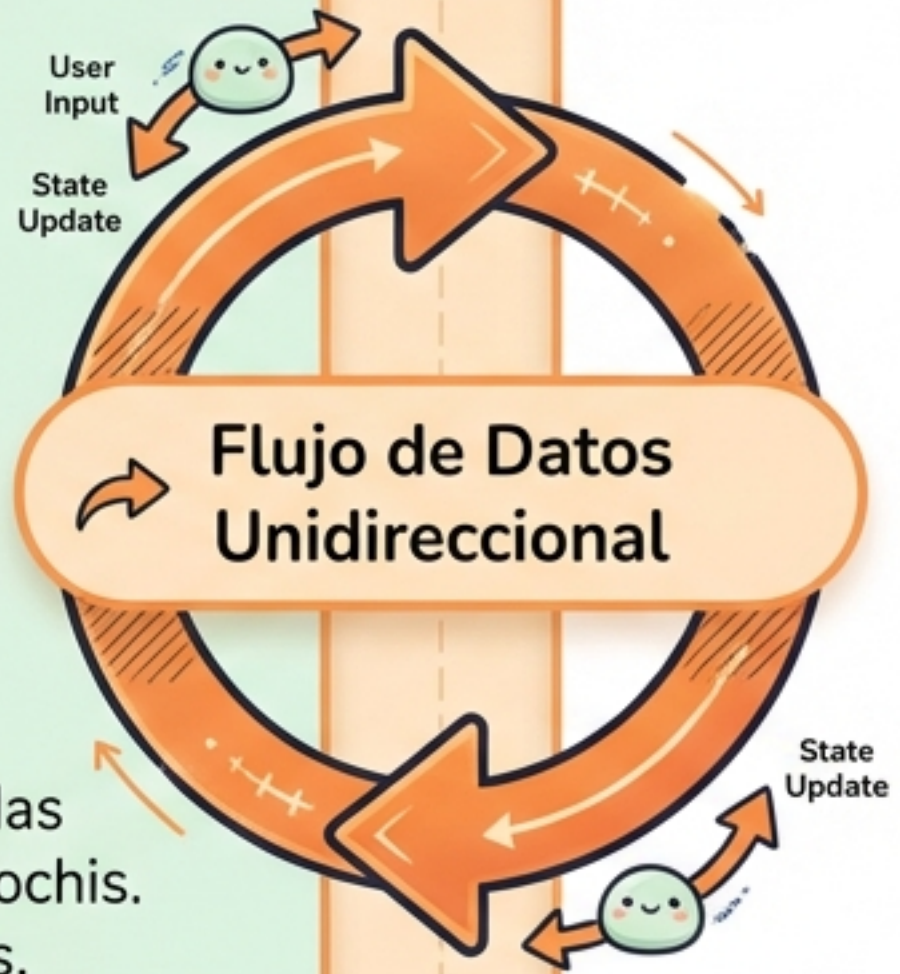


## El Motor



Gestiona los datos puros. Aquí viven las coordenadas, la gravedad y la lista de mochis. No sabe nada de colores ni pantallas.

```
CODE SNIPPET  
var mochis = mutableStateListOf<Mochi>()
```



## La Pantalla



Solo obedece. Lee la lista de datos del **Motor** y **dibuja píxeles** en el Canvas. Captura los toques del usuario y se los envía al **Motor**.



# Anatomía de un Componente Reutilizable

## La Plantilla

```
@Composable  
fun BotonJuego(texto: String, color: Color,  
    alHacerClic: () -> Unit)
```

## El Resultado

Jugar Modo Zen



1

2

3

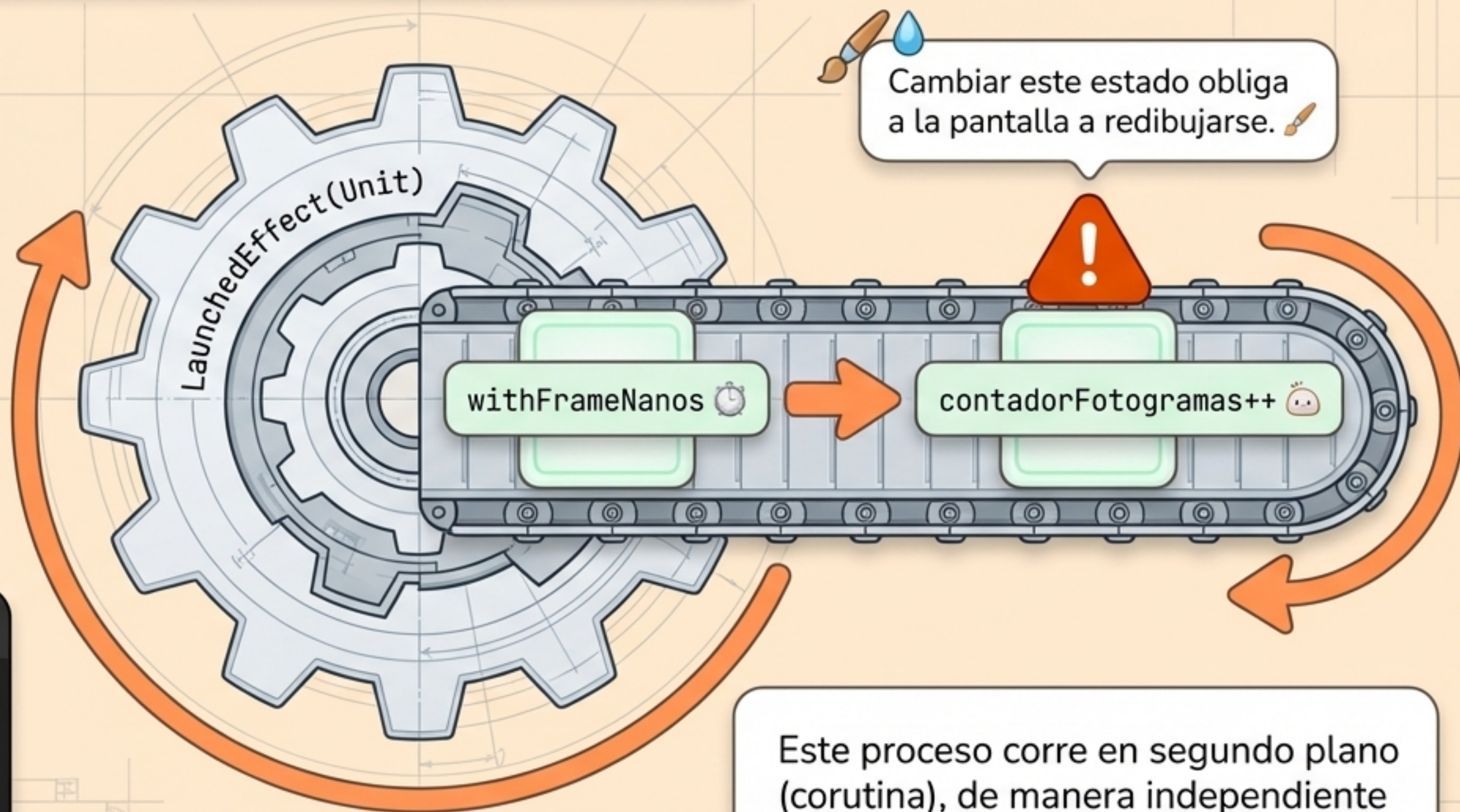
Acción diferida

En lugar de repetir 8 líneas de código por botón en nuestro menú principal, creamos un bloque de construcción reutilizable.





# El Bucle Infinito: El Corazón del Juego ⚙️



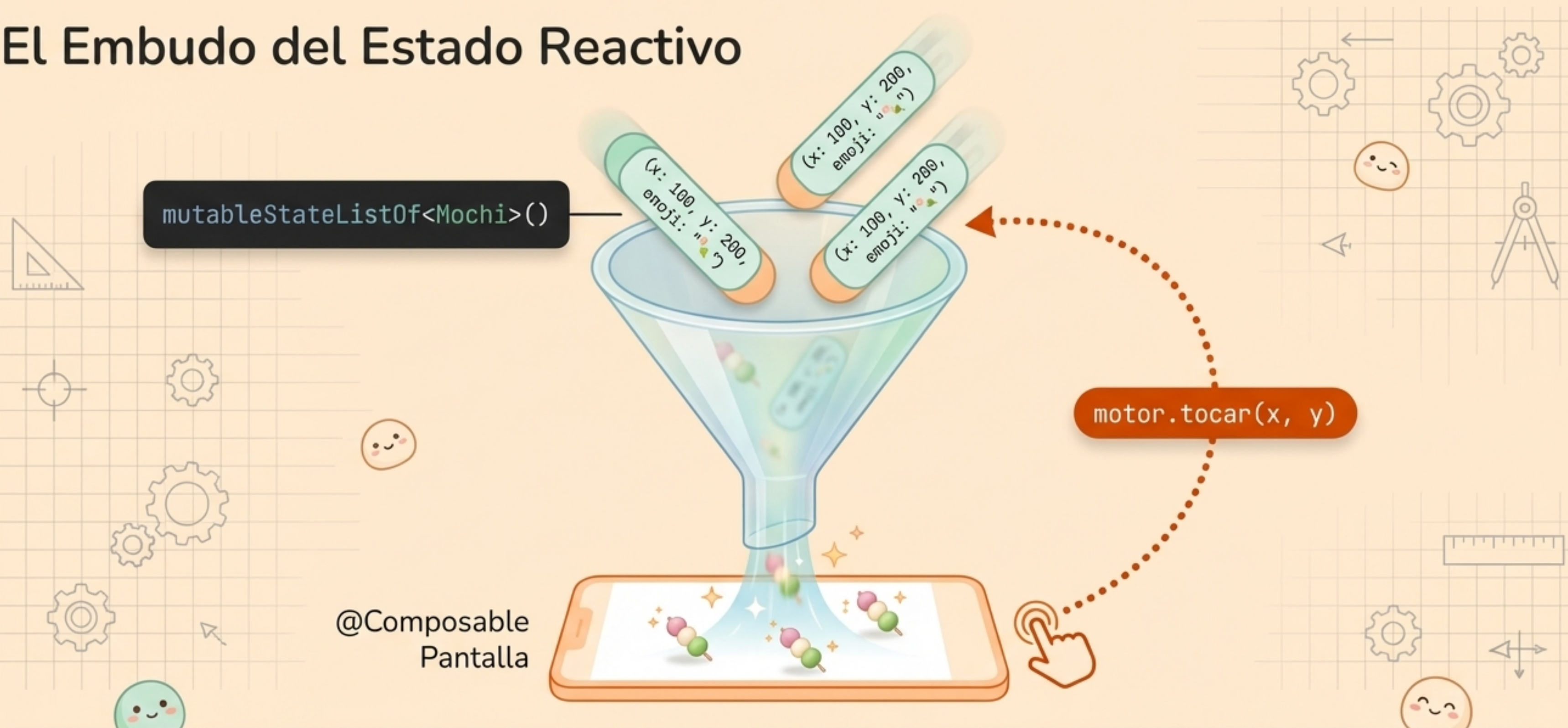
Cambiar este estado obliga a la pantalla a redibujarse. 🖌️

```
while(true) {  
    withFrameNanos {  
        contadorFotogramas++  
    }  
}
```

Este proceso corre en segundo plano (corutina), de manera independiente a los toques del usuario, creando la ilusión del tiempo. ⌚ 🤖



# El Embudo del Estado Reactivo



@Composable  
Pantalla

La pantalla reacciona automáticamente al estado. Si añadimos, modificamos o eliminamos un Mochi del mutableStateList, Compose se encarga de redibujar la pantalla al instante.



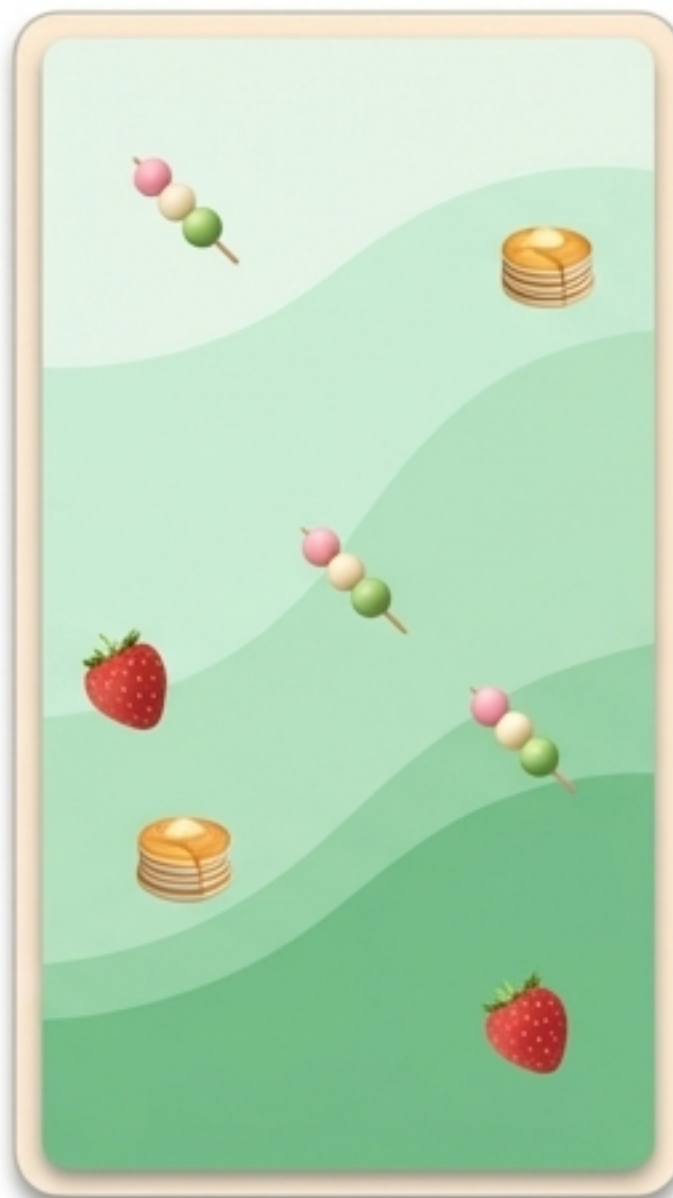
# Nivel 1: El Lienzo Zen (Dibujo Directo)



Aprender a dibujar en la pantalla. Sin físicas, solo creación pura.

MAX\_MOCHIS = 1000

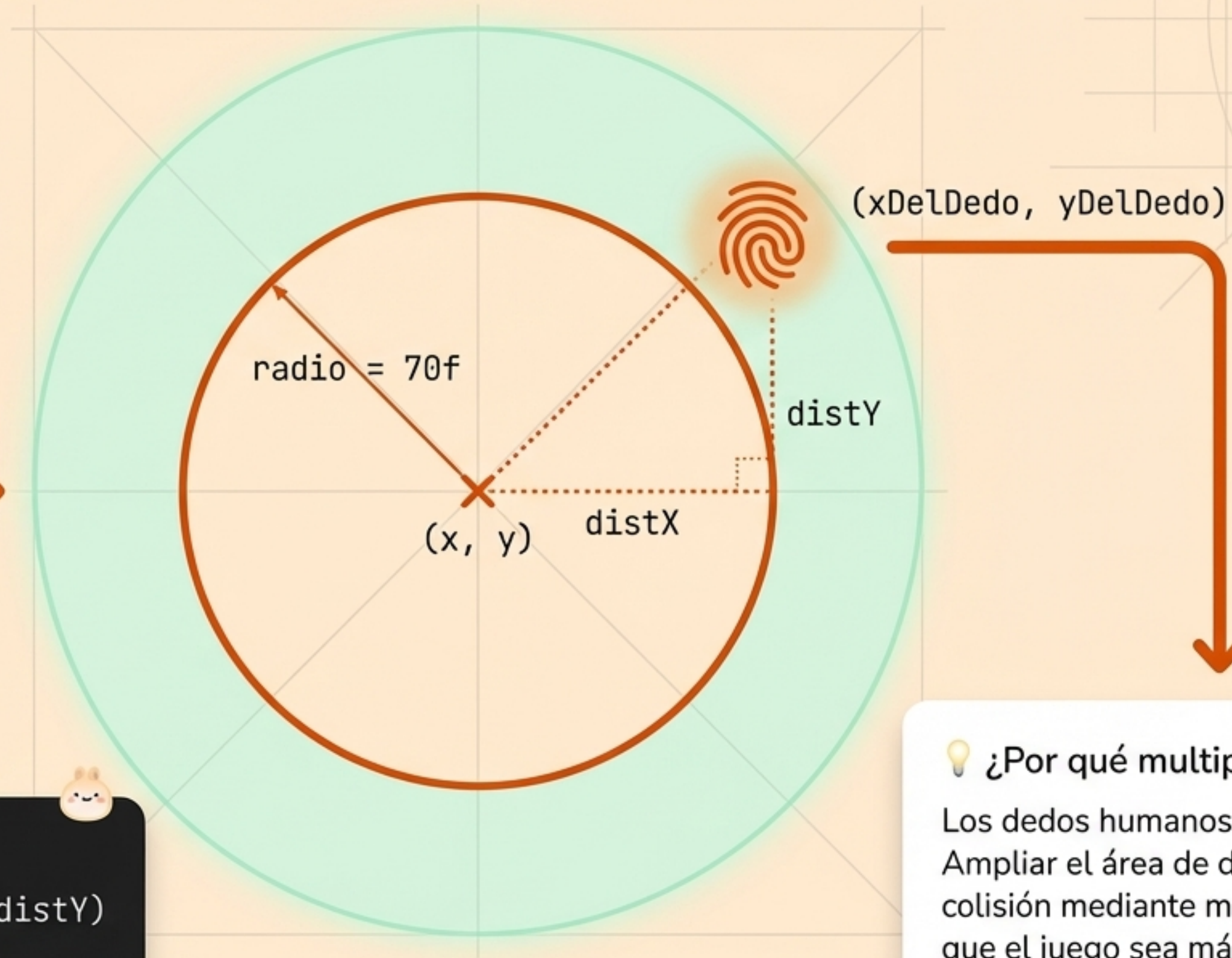
```
fun tocar(x, y) {  
    mochis.add(Mochi(x, y, emoji))  
}
```



Cada Mochi es instanciado con su tiempoCreacion usando `System.currentTimeMillis()`. Un simple toque genera un dato inmutable en esa coordenada.



# Anatomía de un Toque: Matemáticas de Colisión



```
(distX * distX) + (distY * distY)
<= (radio * radio * 1.5f)
```

💡 ¿Por qué multiplicar por 1.5f?

Los dedos humanos son imprecisos. Ampliar el área de detección de colisión mediante matemáticas hace que el juego sea más indulgente y natural.



# Nivel 2: Inyectando Física y Gravedad

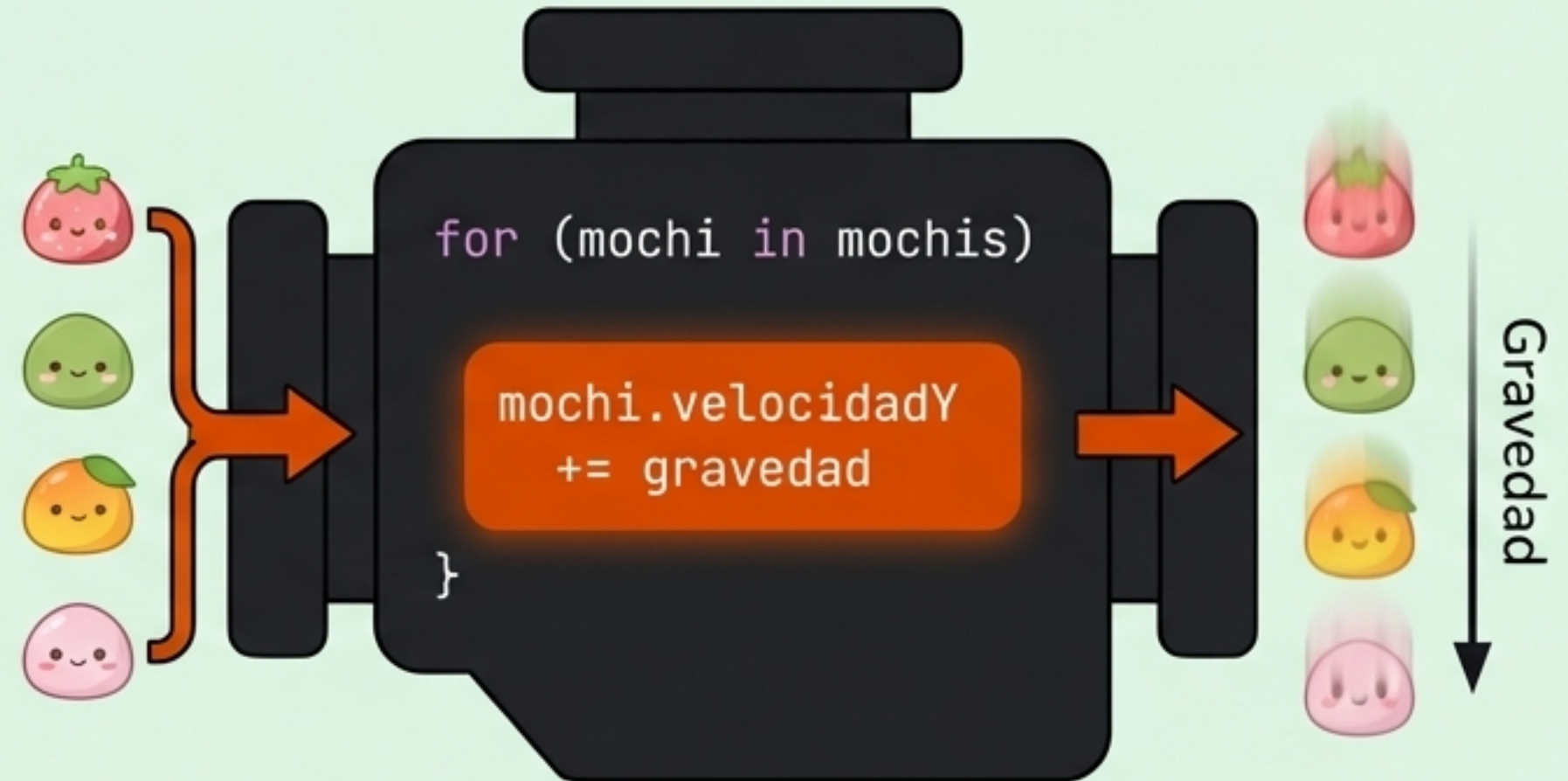
## Reglas del Universo

gravedad = 0.5f  
elasticidad = 0.75f  
friccionSuelo = 0.9f

MAX\_MOCHIS = 100

Calcular colisiones es  
costoso para la CPU 🥲

## El Bucle Matemático



A diferencia del Modo Zen, el Motor Físicas recalcula la posición Y de todos los mochis decenas de veces por segundo, simulando una aceleración constante hacia abajo.



# Las Matemáticas de un Rebote

## Caída



velocidadY  
aumentando

## Impacto



altoPantalla - radio

## El Rebote



```
velocidadY = -velocidadY * elasticidad
```

Al invertir la velocidad (-) cambiamos la dirección. Al multiplicarla por la elasticidad (0.75f), el Mochi pierde un 25% de su energía cinética en cada impacto contra el suelo.



## Nivel 3: Lógica Arcade y Memoria de Estado



```
gravedad = 0.01f
```

En lugar de caer acelerando, los globos experimentan flotabilidad, empujándolos suavemente hacia arriba.



```
explotado = true  
puntuacion++
```

El motor ahora debe recordar qué objetos han sido pinchados para no volver a contarlos y reproducir su animación de explosión.



```
MAX_MOCHIS = 1000
```

El Motor genera enemigos automáticamente sin intervención del usuario, gestionando la memoria y limpiando los que salen de la pantalla.

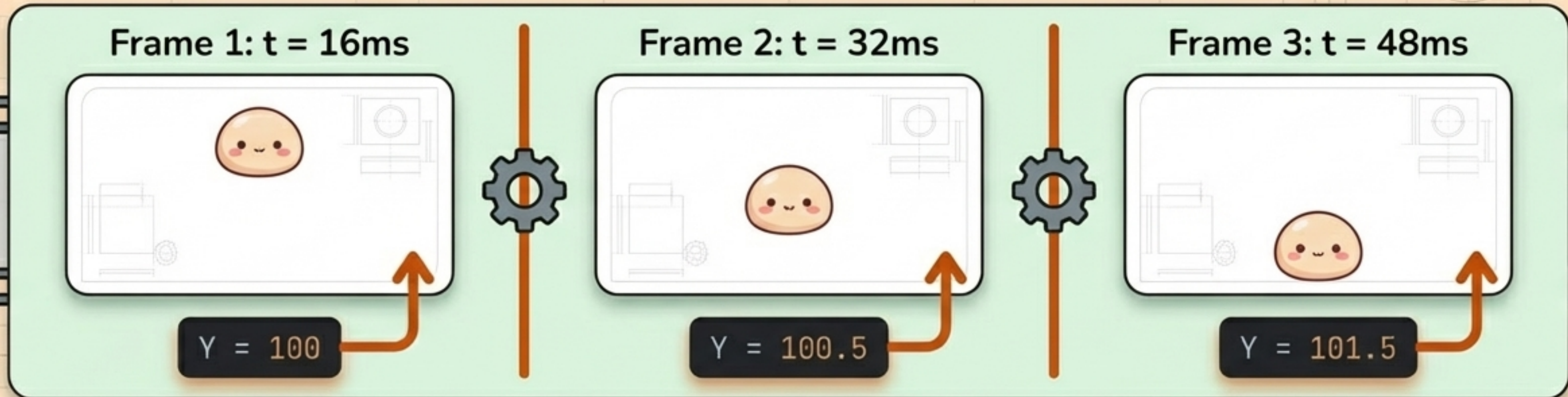


# Tres Motores, Tres Universos

|                            | Motor Zen                 | Motor Físicas               | Motor Arcade                  |
|----------------------------|---------------------------|-----------------------------|-------------------------------|
| Regla Física               | Ninguna<br>(Flotación)    | Gravedad hacia<br>abajo     | Flotabilidad<br>hacia arriba  |
| Interacción<br>Táctil      | Instanciar nuevo<br>Mochi | Aplicar fuerza /<br>Mover   | Cambiar estado<br>a explotado |
| Memoria de<br>Estado       | Inmutable tras<br>crear   | Mutación<br>constante (XYZ) | Mutación y<br>Limpieza        |
| Límite CPU<br>(MAX_MOCHIS) | 1000<br>(Barato)          | 100<br>(Cálculos pesados)   | 1000<br>(Optimizado)          |



# El Espejismo del Movimiento: UI Basada en Estado



El Canvas **jamás** mueve a los Mochis. Es **ciego** y **estático**.  
Toda la ilusión de animación nace del Motor cambiando la matemática en segundo plano y obligando a la Pantalla a redibujarse desde cero en cada fotograma.

**Dominar Jetpack Compose no es dominar la pantalla;  
es dominar los datos que la alimentan.**